ZeroNetPay

Offline UPI · Device-Bound Digital Token Payments

A cryptographic layer on top of UPI that works with zero internet on both sides.
Load once online. Pay offline. Cash out when either side reconnects.

Live demo: https://upizeronetpay.pages.dev
Backend: https://zeronetpayupi.onrender.com
Repo: github.com/Mugilarasi04/zeronetpayupi

1. Executive summary

400 million Indians use UPI daily, yet every payment fails the moment the network drops — metros, tunnels, hill stations, stadiums, rural belts. Existing offline solutions (UPI Lite, CBDC offline, USSD) share one critical flaw: they all require at least one side of the transaction to be online. Nobody has solved true peer-to-peer offline payment.

ZeroNetPay is the offline layer UPI never had. Users link their existing UPI ID once. Money converts into cryptographically-signed digital tokens stored only on the device. Tokens transfer between phones offline via QR (both phones in airplane mode). When any holder returns online, tokens redeem to real money in their bank — with no manual reconciliation.

Why now

- **RBI Regulatory Sandbox Cohort 5** explicitly targets offline / rural payment solutions.
- 600 million Indians live with patchy connectivity — the addressable market is the whole country.
- Real UPI on-ramp + real UPI off-ramp already work via existing bank rails — no NPCI cooperation needed for MVP.
- Cryptographic escrow custody with a mathematical invariant makes the model auditable end-to-end.

2. Live URLs & demo credentials

Component	URL / value
Frontend (PWA)	https://upizeronetpay.pages.dev
Backend (REST API)	https://zeronetpayupi.onrender.com
Escrow UPI ID	mugilarasims@okicici (env-locked)
Escrow WhatsApp	+91 73059 67141 (AKSHAYAKUMAR T)
Health check	GET /api/health → {"ok": true}
Audit invariant	GET /api/audit → escrow == circulating tokens

3. User walkthrough — the 90-second demo

Setup

Two phones (or two Safari tabs): one is the **sender**, one is the **receiver**. Both open https://upizeronetpay.pages.dev once while online so the PWA shell caches. From this point, most steps run offline.

Step 1 — Onboard (~15 seconds each)

- Enter your UPI ID (this is where cashouts land).
- Enter your phone number with country code (used for WhatsApp notifications and complaints).
- Set a 4-6 digit PIN. Biometric (Touch ID / Face ID / fingerprint) auto-enrols on the first payment.
- You are locked out on every tab close until PIN or biometric — no OTP.

Step 2 — Load (only step that needs internet, ~30 sec)

- Tap **Load** → enter amount → the app shows a UPI QR pointing to the escrow account.
- Scan with your real UPI app (GPay / PhonePe / WhatsApp Pay / BHIM) and pay.
- Take a screenshot of the "Payment successful" screen.
- Return to the app → upload the screenshot → tokens are minted to your device.

Step 3 — Offline transfer (airplane mode, ~10 sec)

- Both phones toggle **Airplane Mode ON**.
- Sender: **Pay** tab → amount → biometric → QR appears. Tokens stay in wallet until confirmed sent.
- Receiver: **Receive** tab → camera decodes the QR in one frame → "Payment received".
- Sender: taps **Sent ✓** — tokens leave sender's wallet, appear in receiver's wallet.
- Fallback if QR won't scan: sender shows a **6-digit pair code**, receiver types it in.

Step 4 — Cash out (needs internet again, ~20 sec)

- Turn airplane mode OFF on receiver's phone.
- Cash out tab → enter amount → biometric → server marks tokens spent, creates a payout row.
- App opens WhatsApp with a message to the escrow operator: "Please release my ■X payout, ref STL_..."
- Escrow operator's phone: Disburse view → taps **Pay now** → their UPI app pays receiver → taps **Mark as paid**.
- WhatsApp auto-opens with a "Payout sent, ref XYZ" message to the receiver.
- Receiver's **My cashouts** tab flips from ■pending to ✓paid within 10 seconds.

Step 5 — Recovery paths (built-in)

- **Cascade transfers:** received tokens are spendable — receiver can pay a third person offline; last holder to redeem wins.
- **Self-verify:** if the escrow forgets to mark paid, receiver taps ✓ **I received the money** to close the row.
- **Complaint:** after 1 hour of no payout, receiver taps **Raise complaint** — the escrow operator's dashboard turns red.
- **Double-spend guard:** if the same tokens are replayed, backend returns already_spent — first cashout wins.

4. System workflow & architecture

Data flow

Stage	What happens	Online?
Load	Real UPI payment to escrow → backend mints signed tokens → tokens saved to device IndexedDB.	Yes
Pay (sender)	Wallet selects tokens FIFO → biometric auth → QR frames encode signed payload.	No
Receive	Camera decodes QR (or 6-digit pair code) → receiver's IndexedDB stores tokens.	No
Cascade	Received tokens become spendable — receiver may pay a third party offline.	No
Cash out	Receiver POSTs tokens to /api/redeem → server verifies signature + not-spent + not-expired → creates settle	Yes
Disburse	Escrow operator taps Pay now → real UPI payout to receiver → marked paid.	Yes

Cryptographic invariant

Every token is signed as `HMAC-SHA256(secret, id || value_paise || issued_to_user_id || issued_to_device || issued_at || expires_at)`. Only the backend holds the secret, so tokens are unforgeable. The escrow ledger holds a strict invariant:

escrow_balance == sum(value_paise WHERE spent = 0)

Live at GET /api/audit. If the invariant ever breaks (which it never does under normal operation), the endpoint returns `invariantHolds: false` — a machine-verifiable trust signal.

Security guards (implemented)

Threat	Defence
Fake tokens	HMAC-SHA256 — only backend can sign. Bad signatures rejected at redeem.
Double-spend	First redemption marks spent; second attempt returns already_spent.
Expired tokens	30-day TTL enforced at redeem. Expired tokens can be refunded.
Phone theft / QR reuse	Each payment requires biometric or PIN before the QR is generated.
Session hijack	App locks on every tab close — biometric or PIN to unlock; not cookie-based.
Escrow tampering	ESCROW_LOCKED env freezes the escrow UPI ID; PUT rejected with 403.
Payout dispute	1-hour complaint gate; escrow operator sees red banner + note.

5. Tech stack

Frontend (PWA)

Layer	Choice	Why
Runtime	Vanilla ES modules, no bundler	Zero build step; every browser runs it natively.
Rendering	Hand-rolled component functions	No framework overhead; predictable DOM churn.
Offline shell	Service Worker (cache v25)	App loads with no internet after first visit.
Local storage	IndexedDB (znp DB)	Persists tokens + ledger across app restarts.
QR generation	qrcode-generator (vendored)	Client-side single-frame QR up to 1500 bytes per frame.
QR scanning	jsQR (vendored)	Pure-JS decoder; works offline in the camera loop.
Biometric	Web Authentication API	Real Touch ID / Face ID / fingerprint via platform authenticator.
PIN storage	SHA-256 + salt in localStorage	Lock screen fallback when biometric is unavailable.

Backend (REST API)

Layer	Choice	Why
Runtime	Node.js 20 in Docker on Render	Free tier, GitHub push → auto-deploy.
Framework	Express 4	Small, well understood, express.json body parser.
Database	libsql (Turso-compatible)	Async SQLite driver, no experimental flags, no native compilation.
CORS	cors middleware	Frontend on Cloudflare Pages talks to backend on Render.
Crypto	Node crypto (HMAC-SHA256)	Standard library, no third-party dependency.
Env	dotenv-free custom loader	Reads .env locally; Render's dashboard supplies prod vars.
Rendezvous	In-memory Map for pair codes	5-minute TTL, no persistence needed.

Deployment & ops

Component	Provider	Notes
Domain	Cloudflare Registrar	zeronetpayupi.com (~$9/yr wholesale, WHOIS privacy free).
Frontend hosting	Cloudflare Pages	Free unlimited bandwidth, auto SSL, edge caching.
Backend hosting	Render (free tier)	Docker image built from Dockerfile in repo, GitHub auto-deploy on push.
Version control	GitHub	Public repo — Mugilarasi04/zeronetpayupi.

CI/CD	GitHub → Render + Pages	Zero-config: push to master, both hosts redeploy.
Monitoring	Render logs + /api/audit invariant	One-glance health via the invariant endpoint.

External integrations

- **UPI intent deeplink** (upi://pay?pa=...) — recognized by every UPI app.
- **WhatsApp click-to-chat** (wa.me/<number>?text=...) — receiver ↔ escrow rendezvous, notifications, disputes.
- **PWA install** — Add to Home Screen on iOS/Android, launches without browser chrome.

6. Feature checklist

Feature	Status
UPI ID linking + phone at onboarding	Shipped
4-6 digit PIN + real WebAuthn biometric lock	Shipped
App lock on every tab close (session-only unlock)	Shipped
Real UPI intent QR / deeplink at load time	Shipped
Payment-proof screenshot upload before minting tokens	Shipped
HMAC-SHA256 signed tokens, 30-day TTL	Shipped
Offline QR payment (single-frame for small amounts)	Shipped
6-digit pair-code rendezvous (fallback when QR fails)	Shipped
Full-text transfer-code (advanced fallback)	Shipped
Tokens stay in wallet until sender confirms "Sent ✓"	Shipped
Cascade transfers — received tokens are spendable	Shipped
User-initiated cashout with biometric	Shipped
Escrow-locked to a single UPI ID (env-controlled)	Shipped
Disbursement queue with one-tap Pay now UPI deeplink	Shipped
WhatsApp "nudge escrow" from receiver	Shipped
WhatsApp auto-open on Mark as paid	Shipped
Receiver "✓ I received" self-verify	Shipped
1-hour complaint gate with live countdown	Shipped
Complaint red banner on operator's Home	Shipped
Audit invariant endpoint	Shipped
Double-spend rejection with per-token reason	Shipped
Live URL on custom brand: upizeronetpay.pages.dev	Shipped

7. Anticipated Q & A

Technical

Q. Why HMAC-SHA256 instead of asymmetric signatures?

A. For hackathon speed. HMAC needs no key distribution and every token verifies in one hash. The production upgrade is ECDSA per token: the backend signs with a private key, receivers verify with a cached public key *offline* — same architecture, one primitive swap. We chose HMAC because it made the offline-both-sides claim demonstrable end-to-end in a weekend.

Q. What stops a sender from showing the same QR to two people?

A. Nothing at the QR layer — tokens are bearer instruments (like cash). The **first receiver to redeem online wins**; the second gets rejected with `already_spent`. For high-value flows we would add sender-side sequence numbers stored in a TEE / secure enclave, but for the demo we accept bearer semantics because they are auditable at redeem time.

Q. How does the receiver verify a token offline without server access?

A. They don't cryptographically — they trust the QR envelope and reconcile at cash-out time. If the sender ever forges a token, the receiver discovers it at redeem and can raise a complaint. This is a deliberate simplification of the spec; a production build would use ECDSA so any receiver verifies locally with the backend's public key.

Q. What's the QR payload limit?

A. We cap at 1,500 bytes per QR frame (QR version ~25, error-correction Level L). That fits a ■1–15 payment in a single frame. Larger payments auto-chunk into multiple frames with the receiver's camera reassembling them; a live chip strip shows which frames are still missing.

Q. Why did you replace "I've paid" with a screenshot upload?

A. The old flow was pure honor-system — the user just tapped a button and tokens minted. Judges would rightly call that trust theatre. Screenshot upload creates an auditable paper trail: server stores the base64 image on the load order row before minting. A real production build swaps this for a bank webhook — the interface is the same.

Q. How does the app work offline if it's a web app?

A. Progressive Web App with a Service Worker (cache v25) that pre-caches the entire app shell — HTML, CSS, all JS modules, vendor libs. IndexedDB stores tokens and ledger. Once loaded once, the app runs indefinitely in airplane mode. Add to Home Screen makes it launch like a native app.

Security

Q. What if my phone is stolen with unspent tokens on it?

A. Every spend needs a fresh biometric / PIN via WebAuthn or the SHA-256+salt local hash. A thief without your fingerprint or PIN cannot generate a QR. On top of that, the app locks on every tab close and requires biometric or PIN before showing the wallet. Tokens on-device are useless without live authentication.

Q. How do I know the escrow operator isn't stealing?

A. The /api/audit endpoint publishes the invariant `escrow_balance == tokens_in_circulation`. If the operator withdraws escrow without a matching redemption, the invariant breaks and every user sees it. In production the escrow would be a PPI-licensed bank-partnered account (not the operator's personal UPI), which regulators audit quarterly.

Q. What data does the app send to the server?

A. Only what's necessary: UPI ID + phone + device fingerprint at onboarding, signed tokens at cashout, payment screenshot at load. No location, no contacts, no messages, no analytics. Frontend and backend are open source on GitHub — anyone can audit.

Q. Is the biometric data ever transmitted?

A. No. WebAuthn is a challenge–response protocol; only a signed assertion leaves the device, and even that never leaves the browser in our client-only WebAuthn setup. Actual fingerprint / face data stays in the OS secure enclave forever.

Q. What happens if the backend gets breached?

A. Attacker gets the HMAC secret and can forge tokens. Mitigations already in place: secret is environment-injected (not in git), fresh per deploy via `openssl rand -hex 32`. Production migration to ECDSA moves the private key to an HSM.

Business & regulatory

Q. Who is your competition?

A. UPI Lite, CBDC offline, USSD — all of which require at least one side online at the moment of payment. Our claim is stronger: both sides can be in airplane mode. When NPCI eventually ships official offline UPI for both parties, we become the front-end layer on top of it — the crypto and UX are already built.

Q. What's your business model?

A. Two natural revenue streams. (a) Small merchant subscription for the disbursement dashboard and complaint handling. (b) Interchange share on real UPI settlements routed through our PPI-licensed escrow. Neither is turned on for the demo.

Q. Why should RBI approve this?

A. Because it materially expands UPI's reach into 600 million people with patchy connectivity without breaking any existing rail. Sandbox Cohort 5 is explicitly framed for offline and rural payment innovation — this is exactly the category. Our architecture keeps money in a bank-audited escrow, which is the model RBI already knows how to supervise.

Q. How do you handle KYC?

A. The linked UPI ID is the KYC anchor — every user must already have a bank-KYC'd UPI account to onboard. We add no new KYC surface. For higher transaction limits (■5,000+), we'd trigger PPI-level KYC on top, matching RBI PPI rules.

Q. What are the transaction limits?

A. For the demo, unlimited (■1 minimum). Production would follow RBI PPI Small Wallet limits: ■10,000 per month full-KYC, ■20,000 per month with minimum-KYC upgrade. Escrow itself is capped by the partner bank's PPI license.

Q. How does the money actually move?

A. In: real UPI intent deeplink from user's UPI app to the escrow account (same UPI infrastructure everyone already uses). Out: manual UPI deeplink today (escrow operator taps Pay now to disburse). Production V2: Razorpay/Cashfree payout API for fully automated disbursement.

Q. What's your roadmap?

A. Month 1: this demo. Month 2: incorporate Pvt Ltd. Month 3: apply RBI Sandbox Cohort 5. Months 4-6: sandbox evaluation with real users. Month 6: bank partnership for PPI-licensed escrow. Month 12: full PPI license. Month 18-24: commercial launch.

Product & UX

Q. Why a PWA and not a native app?

A. Distribution and reach. A single URL onboards iPhone, Android, and desktop users. No app store review latency, no per-platform release engineering. If a specific platform requires deeper hardware integration later (e.g. NFC), we'll add a thin native wrapper.

Q. What if the receiver's camera won't scan the QR?

A. Three fallbacks in order of ease. (1) 6-digit pair code — sender's phone posts payload to backend; receiver types 6 digits to fetch it. (2) Full text transfer code — sender copies a long ZNP-XXXX.b64… string, shares via WhatsApp; receiver pastes. (3) Live progress chips on the receive screen show which QR frames are still missing so the receiver knows when to hold the camera steady.

Q. What happens if the tunnel or backend goes down mid-demo?

A. Frontend continues to work for anything cached: showing the wallet, generating QR, scanning QR, cascade transfers. Only load and cashout need the backend. Once the backend recovers, cashout auto-retries silently every 15 seconds. If Render's free tier is asleep, the first request wakes it in ~10 seconds.

Q. How do you keep the receiver notified?

A. Three mechanisms. (a) My-cashouts view polls every 10 sec so paid status shows within 10 sec of the operator marking it. (b) Operator taps Mark as paid → WhatsApp auto-opens on the operator's phone with a "paid, ref XYZ" message to the receiver. (c) Receiver's phone shows a green ✓ paid chip the moment the poll picks it up.

8. Try it now

Open https://upizeronetpay.pages.dev on any phone. Set a PIN. Load ■1. Pay in airplane mode. Cash out. The whole loop takes under two minutes.

Health	https://zeronetpayupi.onrender.com/api/health
Audit	https://zeronetpayupi.onrender.com/api/audit
Settings	https://zeronetpayupi.onrender.com/api/settings
Source	https://github.com/Mugilarasi04/zeronetpayupi

"We didn't build another UPI app. We built the offline layer UPI never had."